

Axel Pestoni, Kyle Matton

Dr. Farzindar

Data Streaming and Mining Algos

4 May 2026

Smart Room Occupancy Monitoring with Clustering Algorithms

Introduction

Energy inefficiencies in modern buildings primarily come from heating, cooling, and lighting systems being active for unoccupied rooms. While traditional occupancy detection systems rely on cameras, this raises serious privacy concerns, with people's images likely being taken without them having any knowledge. This project seeks to address the privacy concern while still having a reliable monitoring system. Our motivation for this project was to apply the skills we had learned from previous courses in machine learning regarding clustering algorithms and apply them in an online setting.

For this project, we used the UCI Occupancy Detection dataset, which contains roughly 8,000 indoor sensor readings from February 2015. These readings contain essential features for identifying occupancy, such as light, CO₂, temperature, humidity, and humidity ratio. Each sensor reading was also assigned a value of occupied or unoccupied, which served as our ground truth. Due to the nature of our data, we needed to reassemble the dataset into a stream so we

could simulate learning in an online environment. To do this, we read the CSV row by row with a 0.01-second delay and created a sliding window with a size of 500, where once the first 500 sensor readings had been recorded, new observations replace the oldest observations. There was also some essential pre-processing work to normalize the CO2 and Light readings by scaling by fixed values ($\text{CO2} \div 2000$ and $\text{Light} \div 1000$)

Algorithms

Streaming K-means

K-means is a centroid-based clustering algorithm that separates a dataset into k distinct groups by iteratively assigning each point to the nearest centroid and then recomputing the centroids to the mean of the points assigned to it. The algorithm starts by selecting k initial centroids, then proceeds through an assignment-and-update step until the iteration is complete. Traditionally, k-means is made to run on a complete dataset, and is not made to accommodate a continuous stream without saving the complete dataset.

To adapt k-means to a streaming setting, we implemented an online variant that maintains exactly $k=2$ centroids (for the two expected states, occupied and unoccupied), and updates incrementally as new readings are processed. Instead of using the entire dataset, the StreamingKmeans class creates the first two centroids from the first two incoming points, then for each subsequent point, it assigns the centroid's position using a running mean ($c \leftarrow (c + (x-c)/n)$), where n is the number of points assigned to the centroid. This update approach requires only a single pass for each point, and uses $O(k)$ memory, satisfying the low-memory focus of streaming algorithms.

Streaming DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups points by local density instead of by distance to a centroid. A point is classified as a *core point* if at least *min_samples* other points are within an ϵ radius of it. Core points that are reachable from one another form a cluster, while points that are not reachable are labeled as noise. Unlike k-means, DBSCAN does not require specifying the number of clusters in advance and can identify arbitrarily shaped clusters, but it requires a full scan of the dataset to build clusters, creating a need for a streaming-specific implementation.

For the streaming setting, we implemented a sliding-window approach in the StreamingDBSCAN class. As each new reading arrives, it is added to a fixed-size deque of 500 points. Once the window is full, the oldest point is automatically removed. Instead of running DBSCAN continuously on each new point, it instead re-fits the DBSCAN on the current window contents every 30-record checkpoint using the parameters $\epsilon = 0.01$ and *min_samples* = 10 on the normalized feature space.

Evaluation Metrics

To evaluate our clustering algorithms, we began by using more traditional metrics like accuracy, precision, recall, and F1 score to determine the quality of clusters that were formed. These are the standard evaluation metrics for most machine learning classification tasks, and we wanted to see initial results from our models to make sure they were functioning as intended. After seeing positive results with these metrics, we began doing further research into more nuanced evaluation metrics for clustering algorithms and decided to go with the silhouette score. The silhouette score helps us better determine the quality of the clusters that were formed by calculating the cohesion and separation for each data point. Cohesion is determined by finding the average distance between a data point and every other data point in the same cluster, and separation is determined by finding the average distance between a point and all the

points in the nearest cluster. Silhouette scores range from -1 to 1, where values closer to 1 indicate well-separated clusters with very minimal overlap, while lower scores indicate very little distinction between the clusters. In addition, we created a confusion matrix to see the number of false positives and false negatives to know if we need to further adjust our algorithms.

Challenges and Solutions

Streaming K-means

Streaming K-means traditionally gives unlabeled cluster indices, which are inherently uninterpretable. In batch settings, the issue is typically resolved by observing the dataset after clustering, but in a streaming context, there is no global view available. To address this, we employ a majority-vote mapping strategy that uses the ground-truth occupancy labels alongside the point assignment as the stream is processed. For each sensor reading, the points' normalized vector reading is assigned to the nearest centroid, and its known occupancy is recorded in a per-cluster vote table. At evaluation time, each cluster is mapped to whichever occupancy class holds the plurality of accumulated votes for that cluster. This allows the model to run fully unsupervised clustering during the stream while still returning binary occupancy predictions at evaluation.

In a setting where the ground truths are no longer available, the model would be initialized and run with a small labeled dataset, and then the clusters would be associated with a certain tag. This works reasonably well as long as the clusters remain stable: if cluster 0 was predominantly labeled 'occupied' during the labelling phase, you can assume that it will continue to represent occupied readings and classify new points to cluster 0 accordingly.

Streaming DBSCAN

A key challenge with the streaming DBSCAN approach is cluster collapse: during extended periods where one occupancy set is entirely missing in the window, DBSCAN produces only a single large cluster, making the majority-vote mapping fail. To address this, the StreamingDBSCAN class maintains running centroids for each occupancy class across the full stream history, called anchor points. At each fit call, *min_samples* copies of each class centroid are injected into the DBSCAN input along with the window contents. As identical points have a distance of zero, which is within the $\epsilon = 0.01$ radius, the anchors form core points, ensuring that both classes are represented in the clustering even when the live window is massively dominated by one class. Anchor points are then excluded from evaluation to help represent the true skew of the data. Noise points are assigned to the nearest cluster centroid at prediction time rather than being discarded, so that all window points are seen in evaluation.

Visualization

Another challenge we had was creating a good visualization of the clusters. From previous ML classes, we had learned Principal Component Analysis (PCA), which uses ML to reduce the dimensionality of a feature set to 2, allowing it to be plotted on 2 axes. When implementing PCA, we found that the resulting graphs did not contain coherent clusters. Instead, it had a few straight diagonal lines that spanned the full graph. After looking more into it, this was due to the CO2 and light features having a very strong linear relationship with occupancy, which explains the straight lines. To address this, we decided to create our plots solely using CO2 and light as the axes since these were the two strongest indicators of occupancy. This allowed us to create a much cleaner visual representation of the clusters.

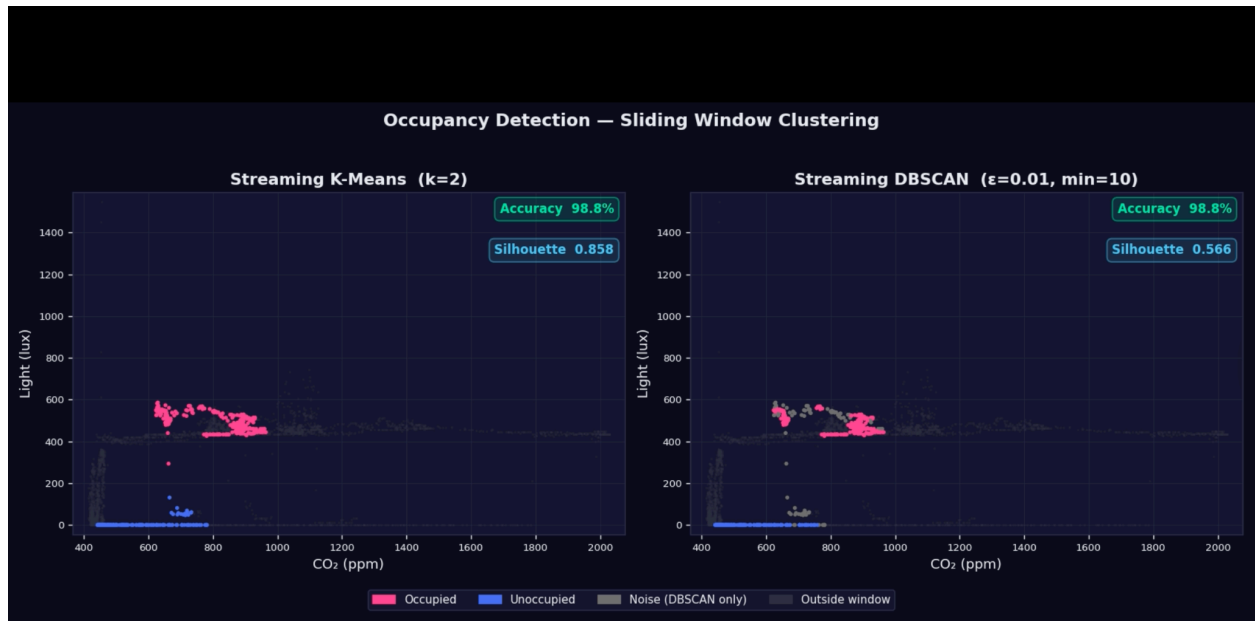
Results

Our models performed very well at clustering the data in an online setting, with the results in the table below:

	Accuracy	Precision	Recall	F1-Score	False Positives	False Negatives	Silhouette Score
K-Means	99.6%	0.9643	1.0000	0.9818	2	0	0.9647
DBSCAN	99.4%	0.9474	1.0000	0.9730	3	0	0.9840

From these results, we can see that both of the models performed very similarly, with the strengths of each being highlighted. The K-Means has a better accuracy, precision, F1-score, and a smaller number of false positives due to the algorithm knowing exactly how many clusters there are from the start, while DBSCAN had to learn that there were only 2 clusters from the density of the distribution in the data. DBSCAN has the edge over K-Means when it comes to cluster quality, with the increased silhouette score due to the ability to handle more nonspherical-shaped clusters.

To get a better visualization of the results, we plotted CO2 vs light and created an animation using Matplotlib of the clusters changing as the sliding window moved with the incoming observations. The full animation can be found in the GitHub repository, but a snapshot from the animation is below:



This animation was done by plotting every data point on this graph with dark gray, and as the sliding window receives the incoming data, it lights up the points with either pink (occupied) or purple (unoccupied). For DBSCAN, the plot has an additional color, light gray (noise), which represents the points that couldn't be classified into a single cluster. By watching the animation, we get a very clear visual when people enter the room because the light will dart up to around 400, and the CO2 will gradually increase along the x-axis. Conversely, we can see when those people then leave the room when the light falls quickly to 0, and we then see the CO2 decreasing as it starts to dissipate.

Individual Contributions

Kyle:	Axel:
- Setting up the dataset into a stream - Initial metrics - Visualizations	- K-Means implementation - DBSCAN implementation - Addressing cluster collapse

Conclusion

Monitoring room occupancy is a vital way to save money by avoiding running expensive systems in rooms that are not currently occupied. Based on the results of this project, we feel that clustering algorithms on non-invasive room data perform incredibly well at classifying occupancy. These algorithms are significantly better than camera systems that record individuals without their consent and are much cheaper computationally since you don't have to analyze images. This lightweight occupancy detection system is much more feasible to roll out to larger buildings where every room needs to be monitored to preserve cost. As the world becomes more reliant on AI systems, it is crucial to implement them in an ethical way.

Sources

Bhattacharjee, Robi, et al. "Online k-means Clustering on Arbitrary Data Streams." *Computer Science*, 2023, <https://cseweb.ucsd.edu/~dasgupta/papers/BIMD23.pdf>. Accessed 4 May 2026.

Shutaywi, Meshal. "Silhouette Analysis for Performance Evaluation in Machine Learning with Applications to Clustering." *MDPI*, 16 June 2021, <https://www.mdpi.com/1099-4300/23/6/759>. Accessed 4 May 2026.

Wang, Daren, et al. "[1706.03113] DBSCAN: Optimal Rates For Density Based Clustering." *arXiv*, 9 June 2017, <https://arxiv.org/abs/1706.03113>. Accessed 4 May 2026.

Wang, Yuechen, et al. "Online-Dynamic-Clustering-Based Soft Sensor for Industrial Semi-Supervised Data Streams." *10.3390/s23031520*, 2022, <https://doi.org/10.3390/s23031520>. Accessed 4 May 2026.